Phase 2 Day 1 – ESP32 IoT Alerting Setup Foundation

1. Objective of the Day

The objective of **Phase 2 Day 1** was to prepare the ESP32 foundation for the IoT alerting stage of the AI Intruder Detection System.

The main focus was not to implement full Wi-Fi or Telegram alerting yet, but to first verify that the ESP32 board is correctly detected, programmed, and able to control basic output devices such as an LED and buzzer.

By the end of the day, the ESP32 was expected to:

```
Detect properly in Arduino IDE
Accept code uploads successfully
Control an external LED
Control a buzzer
Respond to Serial Monitor commands
```

---

2. Work Completed

2.1 ESP32 Board Support Installation

The first task was to install ESP32 board support in the Arduino IDE.

The ESP32 board manager URL was added in:

```
File → Preferences → Additional Boards Manager URLs
```

The following URL was used:

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

After adding the URL, the Boards Manager was opened from:

```
Tools → Board → Boards Manager
```

Then **esp32 by Espressif Systems** was searched and installed successfully.

After installation, ESP32 board options became available in Arduino IDE.

---

3. ESP32 Board and Port Selection

After installing ESP32 board support, the ESP32 was connected to the laptop using a USB cable.

The selected board was:

```
ESP32 Dev Module
```

Initially, the ESP32 COM port was not visible in Arduino IDE. This created a connection issue, because without the COM port the board could not be programmed.

---

4. Challenge Faced: ESP32 COM Port Not Showing

Problem

When the ESP32 was connected to the laptop, the port number was not showing in Arduino IDE under:

```
Tools → Port
```

This indicated that the laptop was not properly detecting the ESP32 USB-to-serial interface.

Cause

The issue was related to the missing USB-to-UART driver. The ESP32 board required a USB bridge driver for proper communication between the laptop and ESP32.

Solution

The **Silicon Labs USB to UART Bridge VCP driver** was installed.

After installing the driver, the ESP32 COM port became visible in Arduino IDE.

The ESP32 was successfully detected on:

```
COM4
```

This confirmed that the driver issue was solved and the laptop could now communicate with the ESP32.

---

5. ESP32 Blink Test

After the ESP32 was detected, a basic blink test was uploaded to verify that the board could accept code and execute it.

The initial blink code used GPIO 2:

```
#define LED_PIN 2

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(1000);

  digitalWrite(LED_PIN, LOW);
  delay(1000);
}
```

The code uploaded successfully. The upload output confirmed that the ESP32 was connected and programmed correctly.

Important upload confirmations included:

```
Connected to ESP32 on COM4
Hash of data verified
Hard resetting via RTS pin...
```

This proved that the ESP32 itself was working properly.

---

6. Challenge Faced: GPIO 2 LED Not Blinking

Problem

Although the code uploaded successfully, the LED did not blink when GPIO 2 was used.

Cause

The onboard LED on this ESP32 board was either not connected to GPIO 2 or was not available as a programmable onboard LED. ESP32 boards can vary depending on manufacturer and model, so the onboard LED pin is not always the same.

Solution

An external LED was connected and tested using a different GPIO pin.

After testing, the LED worked successfully on:

```
GPIO 23
```

So, the LED pin was changed from GPIO 2 to GPIO 23 for the remaining tests.

Updated LED pin:

```
#define LED_PIN 23
```

This solved the LED blinking issue.

---

7. External LED and Buzzer Setup

After verifying that the external LED worked on GPIO 23, the LED and buzzer were connected to the ESP32 for output testing.

Final pin configuration used:

```
LED Pin    → GPIO 23
Buzzer Pin → GPIO 22
GND        → Common Ground
```

The wiring was done as follows:

```
ESP32 GPIO 23 → LED long leg
LED short leg → 220Ω resistor → GND

ESP32 GPIO 22 → Buzzer positive
Buzzer negative → GND
```

A common ground connection was maintained between the LED and buzzer circuit.

---

8. LED and Buzzer Test

The LED and buzzer were tested together using ESP32 GPIO pins.

Test code:

```
#define LED_PIN 23
#define BUZZER_PIN 22

void setup() {
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  digitalWrite(BUZZER_PIN, HIGH);
  delay(1000);

  digitalWrite(LED_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);
  delay(1000);
}
```

Output Observed

The test was successful.

```
LED turned ON and buzzer turned ON for 1 second
LED turned OFF and buzzer turned OFF for 1 second
The cycle repeated continuously
```

This confirmed that both output devices were working properly with the ESP32.

---

9. ESP32 Serial Command Mode Test

After testing continuous LED and buzzer operation, command mode was implemented.

The purpose of command mode was to make the ESP32 respond to Serial Monitor input.

Command logic:

```
Send 1 → LED and buzzer ON
Send 0 → LED and buzzer OFF
```

Command mode code:

```
#define LED_PIN 23
#define BUZZER_PIN 22

void setup() {
  pinMode(LED_PIN, OUTPUT);
```

```
  pinMode(BUZZER_PIN, OUTPUT);

  Serial.begin(9600);
}

void loop() {
  if (Serial.available() > 0) {
    char data = Serial.read();

    if (data == '1') {
      digitalWrite(LED_PIN, HIGH);
      digitalWrite(BUZZER_PIN, HIGH);
    }
    else if (data == '0') {
      digitalWrite(LED_PIN, LOW);
      digitalWrite(BUZZER_PIN, LOW);
    }
  }
}
```

The Serial Monitor was opened and the baud rate was set to:

```
9600
```

Testing was performed by sending commands manually.

Output Observed

When `1` was sent:

```
LED turned ON
Buzzer turned ON
```

When `0` was sent:

```
LED turned OFF
Buzzer turned OFF
```

This confirmed that ESP32 command mode worked successfully.

---

10. Challenges and Solutions

Challenge	Cause	Solution
ESP32 COM port was not showing	Missing USB-to-UART driver	Installed Silicon Labs USB to UART Bridge VCP driver
GPIO 2 LED was not blinking	ESP32 board LED pin variation / onboard LED not mapped to GPIO 2	Used external LED on GPIO 23
Needed reliable output pins	Original test pins were changed after practical testing	Final LED pin set to GPIO 23 and buzzer pin set to GPIO 22

---

11. Final Pin Configuration

The final working ESP32 pin configuration for Phase 2 Day 1 was:

```
LED    → GPIO 23
Buzzer → GPIO 22
GND    → Common Ground
```

This pin configuration will be used for the next ESP32 IoT alerting steps.

---

12. Result

Phase 2 Day 1 was completed successfully.

The following tasks were achieved:

```
ESP32 board support installed in Arduino IDE
ESP32 detected successfully after driver installation
ESP32 connected on COM4
Code upload verified successfully
External LED tested successfully on GPIO 23
Buzzer tested successfully on GPIO 22
LED and buzzer tested together
ESP32 Serial Monitor command mode tested successfully
```

---

13. Conclusion

Phase 2 Day 1 successfully established the ESP32 foundation for IoT alerting.

Although two challenges were faced, both were solved practically. The ESP32 COM port issue was fixed by installing the Silicon Labs USB to UART Bridge VCP driver. The LED issue was solved by changing the LED output pin from GPIO 2 to GPIO 23.

The ESP32 is now able to control the LED and buzzer and respond to Serial Monitor commands. This confirms that the hardware foundation is ready for the next stage.